

Lab 6: Matching, Weighting, and Doubly Robust Estimation

SOCIOL 723, Social Statistics II, Thursday, October 1

Wenhao Jiang

Table of contents

1 The question

Estimand. The average difference in log personal earnings between completing a bachelor’s degree and not completing one, among the full-time workers aged 25–64 in our GSS analytic sample, 2010–2022.

Note

Note the deliberately modest population. The GSS is a complex sample, and doing this properly for the U.S. population would require the survey weights (`wt`) and PSU/stratum-aware inference. We work unweighted here so that the adjustment methods are visible on their own; that is a simplification, and you should say so if you write it up.

We also exclude `hours` from the adjustment set. Hours worked is plausibly a *consequence* of completing college, so conditioning on it would be the post-treatment error from Week 5; it would answer a different question.

This is an *observational* question with obvious confounding: people who complete college differ from those who do not in family background, measured ability, and much else. We are going to assume conditional ignorability given the covariates we have, do the adjustment five different ways, and then ask how much an unobserved confounder would have to matter to overturn the answer.

Warning

Assuming conditional ignorability here is almost certainly wrong. That is the point of the last section. Treat today’s numbers as an exercise in *method*, not as an estimate of the return to college.

```

library(dplyr)
library(ggplot2)
library(MatchIt)
library(cobalt)
library(sensemakr)
library(sandwich); library(lmtest)
library(boot)

gss <- readRDS("../Data/gss_earnings.rds") |>
  mutate(D = ba, Y = llearn)

covs <- c("pareduc", "wordsum", "female", "black", "otherace", "exper")
c(n = nrow(gss), treated = sum(gss$D), control = sum(1 - gss$D))
#>      n treated control
#>   3509    1441    2068

```

2 Where we start: the naive comparison

```

naive <- lm(Y ~ D, data = gss)
reg <- lm(Y ~ D + pareduc + wordsum + female + black + otherace + exper +
  I(exper^2), data = gss)

c(naive = coef(naive)["D"], regression = coef(reg)["D"])
#>      naive.D regression.D
#>      0.6834      0.6295

```

Adjustment moves the estimate substantially. But regression is only one of the adjustment strategies available, and it makes assumptions we have not examined.

```

bal.tab(reg_formula <- D ~ pareduc + wordsum + female + black + otherace +
  exper, data = gss, un = TRUE, thresholds = c(m = 0.1))
#> Balance Measures
#>      Type Diff.Un      M.Threshold.Un
#> pareduc Contin.  0.736 Not Balanced, >0.1
#> wordsum Contin.  0.793 Not Balanced, >0.1
#> female  Binary  0.055   Balanced, <0.1
#> black   Binary -0.053   Balanced, <0.1
#> otherace Binary -0.016   Balanced, <0.1
#> exper   Contin. -0.420 Not Balanced, >0.1
#>
#> Balance tally for mean differences
#>      count

```

```

#> Balanced, <0.1      3
#> Not Balanced, >0.1  3
#>
#> Variable with the greatest mean difference
#> Variable Diff.Un      M.Threshold.Un
#> wordsum    0.793 Not Balanced, >0.1
#>
#> Sample sizes
#>      Control Treated
#> All      2068      1441

```

The `Diff.Un` column is the standardized mean difference. Values well above 0.1, here `pareduc` and `wordsum` especially, mean the two groups are not comparable as they stand.

3 Exact matching on cells

The lecture's matching section began with the cleanest estimator there is: coarsen the covariates into cells, take the difference in means inside every cell that holds both graduates and non-graduates, and average the cell differences with the weights of the estimand you named. This section reproduces the lecture's numbers by hand, then checks them against `MatchIt`.

3.1 Building the cells

```

## coarsen the three continuous covariates at rough terciles; keep the binaries
as they are
gss <- gss |>
  mutate(pareduc_b = cut(pareduc, c(-Inf, 11, 13, Inf), labels = c("<12",
    "12-13", "14+")),
    wordsum_b = cut(wordsum, c(-Inf, 5, 7, Inf), labels = c("0-5",
    "6-7", "8-10")),
    exper_b = cut(exper, c(-Inf, 16, 29, Inf), labels = c("0-16",
    "17-29", "30+")),
    cell = interaction(female, black, pareduc_b, wordsum_b, exper_b))
## one row per cell: how many treated and untreated, and their mean outcomes
cells <- gss |>
  group_by(cell) |>
  summarise(n1 = sum(D), n0 = sum(1 - D),
    m1 = mean(Y[D == 1]), m0 = mean(Y[D == 0]), .groups = "drop") |>
  mutate(n = n1 + n0, tau = m1 - m0, both = n1 > 0 & n0 > 0)
c(possible = nlevels(gss$cell), occupied = nrow(cells), both_arms =
sum(cells$both),
  treated_only = sum(cells$n1 > 0 & cells$n0 == 0), untreated_only =
sum(cells$n0 > 0 & cells$n1 == 0))

```

```
#>      possible      occupied      both_arms      treated_only      untreated_only
#>          108          106           92             2             12
## coverage: who is lost when the one-armed cells are dropped
c(treated_dropped = sum(cells$n1[!cells$both]), untreated_dropped =
sum(cells$n0[!cells$both]))
#>      treated_dropped      untreated_dropped
#>              4              70
```

3.2 The three estimands, and what OLS returns

```
m <- filter(cells, both)
## ATE: cell shares among matched units; ATT: shares among matched treated;
ATU: among matched untreated
est_hand <- c(ATE = sum(m$tau * m$n) / sum(m$n),
              ATT = sum(m$tau * m$n1) / sum(m$n1),
              ATU = sum(m$tau * m$n0) / sum(m$n0))
c(naive = mean(gss$Y[gss$D == 1]) - mean(gss$Y[gss$D == 0]),
  est_hand,
  ## OLS saturated in the cells: the variance-weighted average from the
  lecture
  OLS_cells = coef(lm(Y ~ D + cell, data = gss))["D"])
#>      naive      ATE      ATT      ATU      OLS_cells.D
#>      0.6834      0.6346      0.6247      0.6417      0.6305
```

The four adjusted numbers are the same 92 cell differences under four sets of weights.

3.2.1 The standard error by hand

The lecture's formula: the estimate is a weighted sum of cell differences, so its variance is the weighted sum of the cells' variances, $\text{Var}(\widehat{\text{ATE}}) = \sum_x (n_x/n)^2 (s_{1x}^2/n_{1x} + s_{0x}^2/n_{0x})$, with the ATT and ATU weights in place of n_x/n .

```
## within-cell variances of Y in each arm (NA when a cell has one unit in that
arm)
v1 <- tapply(gss$Y[gss$D == 1], gss$cell[gss$D == 1], var)
v0 <- tapply(gss$Y[gss$D == 0], gss$cell[gss$D == 0], var)
m$v1 <- v1[as.character(m$cell)]; m$v0 <- v0[as.character(m$cell)]
m$v1[is.na(m$v1)] <- 0; m$v0[is.na(m$v0)] <- 0
## the variance of one cell difference, then the weighted sum with each
target's weights
cell_var <- m$v1 / m$n1 + m$v0 / m$n0
se_hand <- c(ATE = sqrt(sum((m$n / sum(m$n))^2 * cell_var)),
            ATT = sqrt(sum((m$n1 / sum(m$n1))^2 * cell_var)),
```

```

      ATU = sqrt(sum((m$n0 / sum(m$n0))^2 * cell_var)))
round(se_hand, 4)
#>      ATE      ATT      ATU
#> 0.0348 0.0388 0.0404

```

Now the bootstrap: resample units, rebuild the cells, recompute all three. Resampling is valid here because the estimate is a smooth function of cell means; nothing is nearest-neighbour.

```

boot_exact <- function(d) {
  b <- d[sample(nrow(d), replace = TRUE), ]
  t <- b |> group_by(cell) |>
    summarise(n1 = sum(D), n0 = sum(1 - D), y1 = sum(Y * D), y0 = sum(Y * (1 -
      D)), .groups = "drop") |>
    filter(n1 > 0, n0 > 0) |> mutate(tau = y1 / n1 - y0 / n0)
  c(ATE = sum(t$tau * (t$n1 + t$n0)) / sum(t$n1 + t$n0),
    ATT = sum(t$tau * t$n1) / sum(t$n1),
    ATU = sum(t$tau * t$n0) / sum(t$n0))
}
set.seed(723)
se_boot <- apply(replicate(200, boot_exact(gss)), 1, sd)
round(rbind(formula = se_hand, bootstrap = se_boot), 4)
#>      ATE      ATT      ATU
#> formula  0.0348 0.0388 0.0404
#> bootstrap 0.0330 0.0400 0.0395

```

The two rows agree to within a couple of thousandths: the formula is doing what the bootstrap does, without the 200 refits.

3.3 The same thing in MatchIt, with standard errors

`method = "exact"` builds the same cells; `estimand` chooses the weights ("ATC" is MatchIt's name for the ATU). `match.data()` returns the matched units with a `weights` column, and a weighted regression of `Y` on `D` is a difference of weighted means, so its coefficient reproduces the by-hand estimate exactly. For the standard error use a sandwich (`vcovHC`): the default `lm` standard error ignores the weights.

```

mi <- list()
for (est in c("ATE", "ATT", "ATC")) {
  mm <- matchit(D ~ female + black + pareduc_b + wordsum_b + exper_b,
    data = gss, method = "exact", estimand = est)
  md <- match.data(mm)
  f <- lm(Y ~ D, data = md, weights = weights)
  mi[[est]] <- c(estimate = unname(coef(f)["D"]),
    se_HC1 = unname(sqrt(vcovHC(f, type = "HC1")["D", "D"])))
}

```

```

}
mi_tab <- do.call(rbind, mi); rownames(mi_tab) <- c("ATE", "ATT", "ATU")
## by hand (section 3.2) next to MatchIt + WLS + sandwich
round(cbind(hand_estimate = c(est_hand["ATE"], est_hand["ATT"],
est_hand["ATU"]),
          matchit_estimate = mi_tab[, "estimate"],
          hand_se = se_hand, matchit_se_HC1 = mi_tab[, "se_HC1"]), 4)
#>      hand_estimate matchit_estimate hand_se matchit_se_HC1
#> ATE      0.6346      0.6346 0.0348      0.0385
#> ATT      0.6247      0.6247 0.0388      0.0406
#> ATU      0.6417      0.6417 0.0404      0.0465

```

The estimates are identical. The sandwich standard errors sit a little above the by-hand ones: the sandwich measures each unit's deviation from its *arm*'s weighted mean, so it counts between-cell variation in Y as noise, whereas the formula uses only within-cell variances. Both treat the cell weights as fixed.

3.3.1 The balance check

`love.plot()` draws the standardized mean difference of every covariate before and after matching. The variables we matched on are the coarsened bins, which balance exactly by construction; `add1` adds the underlying continuous variables, where the within-band imbalance shows, and `otherace`, which we did not match on at all.

```

mm_ate <- matchit(D ~ female + black + pareduc_b + wordsum_b + exper_b,
                  data = gss, method = "exact", estimand = "ATE")
love.plot(mm_ate, data = gss, add1 = ~ pareduc + wordsum + exper + otherace,
          thresholds = c(m = 0.1), binary = "std",
          colors = c("firebrick", "navy")) +
  theme_minimal(base_size = 9)

```

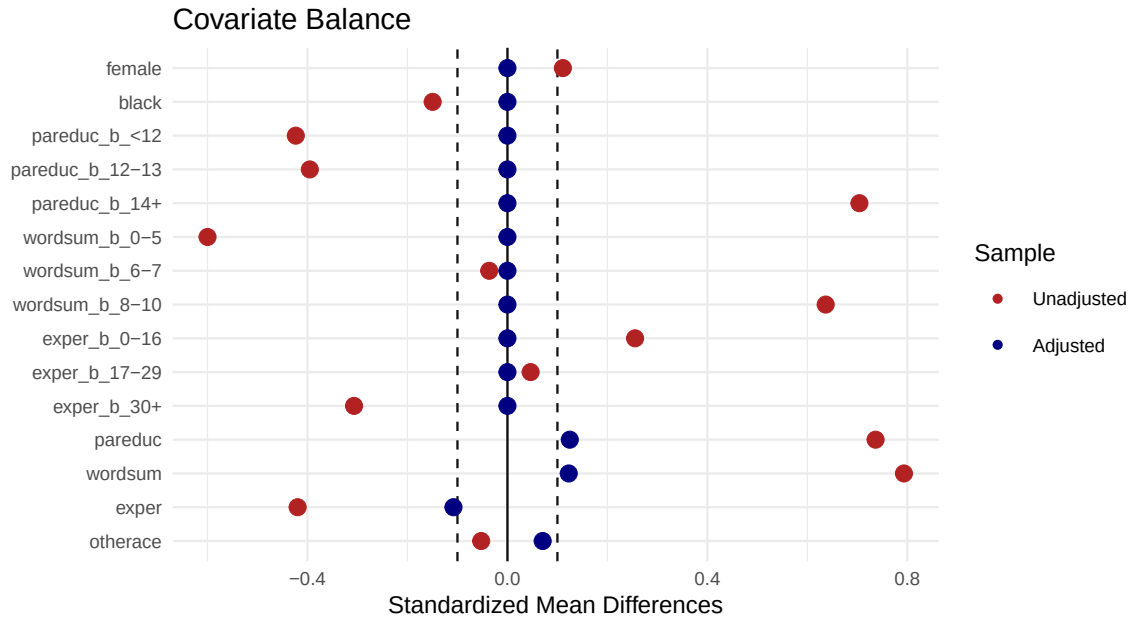


Figure 1: Covariate balance before and after exact matching on the cells (ATE weights). The bins balance exactly; the continuous variables keep the within-band imbalance the bins allow.

4 The propensity score

```
## a logit for  $Pr(D = 1 \mid X)$ ; family = binomial("logit") says so explicitly
## (binomial() alone would also default to the logit link, but say what you
## mean)
ps_mod <- glm(D ~ pareduc + wordsum + female + black + otherace +
              exper + I(exper^2),
              data = gss, family = binomial("logit"))
gss$ps <- fitted(ps_mod)

summary(gss$ps)
#>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#> 0.00265 0.21792 0.39107 0.41066 0.59004 0.94567
```

```
ggplot(gss, aes(ps, fill = factor(D))) +
  geom_density(alpha = .45, colour = NA) +
  scale_fill_manual(values = c("0" = "navy", "1" = "firebrick"),
                    labels = c("no BA", "BA"), name = NULL) +
  labs(x = "estimated propensity score", y = "density") +
  theme_minimal(base_size = 10)
```

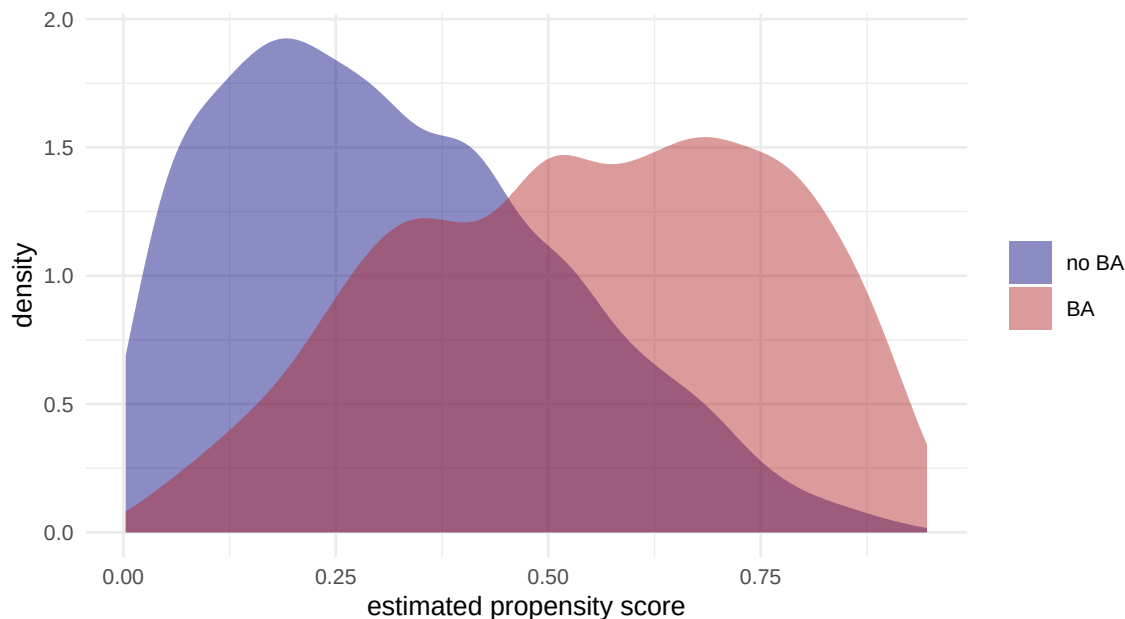


Figure 2: Distribution of the estimated propensity score by treatment status. Overlap is imperfect in both tails.

```
## how many treated units sit where controls are scarce, and vice versa?
c(treated_above_0.9 = sum(gss$D == 1 & gss$ps > 0.9),
  controls_above_0.9 = sum(gss$D == 0 & gss$ps > 0.9),
  controls_below_0.1 = sum(gss$D == 0 & gss$ps < 0.1),
  treated_below_0.1 = sum(gss$D == 1 & gss$ps < 0.1))
#> treated_above_0.9 controls_above_0.9 controls_below_0.1 treated_below_0.1
#>                34                3                305                28
```

i Note

Note the trade-off from lecture: a propensity model that predicts treatment *well* is a model that separates the groups, which is exactly the situation in which comparison is hardest. Judge the model by **balance**, not by AUC.

5 Matching

5.1 Nearest-neighbour on the propensity score

Each treated unit takes the control with the closest propensity score. **With replacement** (`replace = TRUE`) the same control may serve several treated units: every treated unit gets its best match, so balance is far better, and the price is the reuse counts K_j from lecture, which inflate the variance and rule out the ordinary bootstrap. Without replacement, once the good controls are used up the remaining treated units get poor matches.


```
m_nn <- matchit(D ~ pareduc + wordsum + female + black + otherace +
  exper + I(exper^2),
  data = gss, method = "nearest", distance = "glm",
  estimand = "ATT", replace = TRUE)

m_nn
#> A `matchit` object
#> - method: 1:1 nearest neighbor matching with replacement
#> - distance: Propensity score
#> - estimated with logistic regression
#> - number of obs.: 3509 (original), 2150 (matched)
#> - target estimand: ATT
#> - covariates: pareduc, wordsum, female, black, otherace, exper, I(exper^2)
```

```
love.plot(m_nn, thresholds = c(m = 0.1), binary = "std",
  colors = c("firebrick", "navy")) +
  theme_minimal(base_size = 9)
```

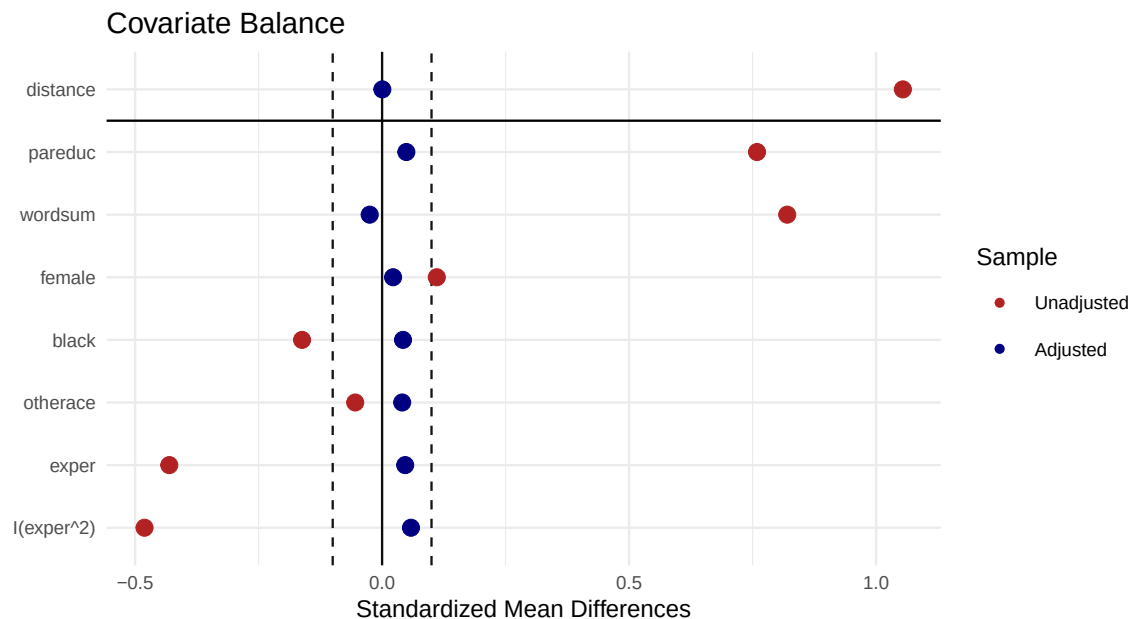


Figure 3: Covariate balance before and after nearest-neighbour matching on the propensity score, with replacement. Everything lands inside the dashed lines.

```
## the same matching without replacement: 1,441 treated compete for 2,068
controls
## and the late ones get poor matches. Compare the adjusted standardized
differences.
m_nn0 <- matchit(D ~ pareduc + wordsum + female + black + otherace +
  exper + I(exper^2),
```

```

        data = gss, method = "nearest", distance = "glm",
        estimand = "ATT", replace = FALSE)
round(cbind(with_replacement = bal.tab(m_nn)$Balance$Diff.Adj,
          without           = bal.tab(m_nn0)$Balance$Diff.Adj)[-1, ], 3)
#>      with_replacement without
#> [1,]          0.049   0.413
#> [2,]         -0.025   0.439
#> [3,]          0.011   0.027
#> [4,]          0.014  -0.049
#> [5,]          0.012   0.008
#> [6,]          0.047  -0.210
#> [7,]          0.058  -0.208

```

```

## get_matches() returns one row per matched pair member, so a reused control
## appears once per treated unit it serves; `unit` identifies the person,
## `subclass` the matched pair. The data already have a column called id,
## hence the id = "unit" argument.
md_nn <- get_matches(m_nn, id = "unit")
c(rows = nrow(md_nn), distinct_controls = length(unique(md_nn$unit[md_nn$D ==
0])),
  max_reuse = max(table(md_nn$unit[md_nn$D == 0]))))
#>      rows distinct_controls      max_reuse
#>      2882             709             23
fit_nn <- lm(Y ~ D + pareduc + wordsum + female + black + otherace +
  exper + I(exper^2),
  data = md_nn, weights = weights)
## standard errors clustered on both the pair and the person (MatchIt's
recommendation
## for matching with replacement); the formal answer is the Abadie-Imbens
variance
## from lecture, and the ordinary bootstrap is NOT valid here
round(coeftest(fit_nn, vcov = vcovCL, cluster = ~ subclass + unit)["D", ], 4)
#> Estimate Std. Error   t value Pr(>|t|)
#>    0.6342    0.0462   13.7231   0.0000

```

Two things to notice. First, we still run a regression on the matched data: matching is *preprocessing*, and the regression mops up residual imbalance. Second, the standard errors are clustered on the matched pair and on the person, because a control reused K_j times appears K_j times.

The lecture's variance formula, with the K_j^2 term, is what `Matching::Match()` reports (`install.packages("Matching")`). It matches on the same propensity score and prints the Abadie-Imbens standard error directly, with no regression afterwards.

```

library(Matching)
## one-to-one nearest neighbour on the propensity score, with replacement,
ATT:
## the same design as m_nn above; m_nn$distance is MatchIt's estimated
propensity score.
## Match() keeps every control tied at the closest distance, at equal weight
(MatchIt picks
## one); the GSS has many identical covariate profiles, so ties are common
here
mm <- Match(Y = gss$Y, Tr = gss$D, X = m_nn$distance, M = 1, estimand = "ATT",
            replace = TRUE)
summary(mm)
#>
#> Estimate... 0.63758
#> AI SE..... 0.049562
#> T-stat..... 12.864
#> p.val..... < 0.000000000000000222
#>
#> Original number of observations..... 3509
#> Original number of treated obs..... 1441
#> Matched number of observations..... 1441
#> Matched number of observations (unweighted). 5208
## how many distinct controls were used, and the largest K_j
c(distinct_controls = length(unique(mm$index.control)),
  max_reuse = max(table(mm$index.control)))
#> distinct_controls      max_reuse
#>                1668          23

```

The AI SE line is the Abadie-Imbens standard error: it counts a control used K_j times K_j^2 times, which is why it is larger than a naive SE that treats the matched rows as independent. It is close to the clustered SE above. The matched sample is larger than MatchIt's because tied controls are all kept; the estimate barely moves. (With `ties = FALSE` Match() drops the Abadie-Imbens calculation and prints a naive SE, so leave the default.)

5.2 Mahalanobis distance

Instead of collapsing the covariates to a propensity score, match on the covariates directly, scaling by their covariance so that no variable dominates through its units.

```

m_mah <- matchit(D ~ pareduc + wordsum + female + black + otherace +
                 exper,
                 data = gss, method = "nearest", distance = "mahalanobis",
                 estimand = "ATT")

```

```
summary(m_mah)$nn
#>           Control Treated
#> All (ESS)      2068    1441
#> All           2068    1441
#> Matched (ESS)  1441    1441
#> Matched       1441    1441
#> Unmatched      627      0
#> Discarded      0       0
```

5.3 A caliper

A caliper refuses matches beyond a set distance on the propensity score, measured in standard deviations of the linear predictor.

```
m_cal <- matchit(D ~ pareduc + wordsum + female + black + otherace +
  exper + I(exper^2),
  data = gss, method = "nearest", distance = "glm",
  estimand = "ATT", caliper = 0.1, std.caliper = TRUE)
summary(m_cal)$nn
#>           Control Treated
#> All (ESS)      2068    1441
#> All           2068    1441
#> Matched (ESS)  1017    1017
#> Matched       1017    1017
#> Unmatched     1051     424
#> Discarded      0       0
```

Look at the **Unmatched** row for the treated. Those respondents have no comparable counterpart in the data, and dropping them silently redefines the estimand; you are now describing the subpopulation of degree-holders who have plausible non-degree counterparts.

5.4 Coarsened exact matching

```
## MatchIt's default coarsening: each continuous variable is cut into bins by
## Sturges' rule (about log2(n) + 1 of them), binaries as they are
m_cem <- matchit(D ~ pareduc + wordsum + female + black + exper,
  data = gss, method = "cem", estimand = "ATT")
summary(m_cem)$nn
#>           Control Treated
#> All (ESS)      2068.0    1441
#> All           2068.0    1441
#> Matched (ESS)   521.1     843
#> Matched       1119.0     843
```

```

#> Unmatched      949.0      598
#> Discarded       0.0       0
round(bal.tab(m_cem, un = TRUE)$Balance[, c("Diff.Un", "Diff.Adj")], 3)
#>      Diff.Un Diff.Adj
#> pareduc    0.759   -0.008
#> wordsum    0.820    0.000
#> female     0.055    0.000
#> black     -0.053    0.000
#> exper     -0.431    0.000

```

CEM balances the *coarsened* variables exactly, and nothing more: how much imbalance remains on the underlying continuous variables depends entirely on how wide the bins are. With MatchIt's default bins, which are narrow, the residual differences above are essentially zero, but look at the **Unmatched** row: 598 of the 1,441 treated units found no control in their bin and were dropped, so the estimand is now the effect on the matchable treated. Section 3 used three wide bins per variable and lost 4 treated units, at the price of standardized differences near 0.1 on the continuous variables. The same trade-off, chosen by the bins:

```

## our section-3 bins, handed to MatchIt as cutpoints
m_cem3 <- matchit(D ~ pareduc + wordsum + female + black + exper,
                  data = gss, method = "cem", estimand = "ATT",
                  cutpoints = list(pareduc = c(11.5, 13.5),
                                   wordsum = c(5.5, 7.5),
                                   exper   = c(16.5, 29.5)))

summary(m_cem3)$nn
#>      Control Treated
#> All (ESS)      2068   1441
#> All            2068   1441
#> Matched (ESS)   685   1437
#> Matched        1998   1437
#> Unmatched       70     4
#> Discarded       0     0
round(bal.tab(m_cem3)$Balance[, "Diff.Adj", drop = FALSE], 3)
#>      Diff.Adj
#> pareduc    0.145
#> wordsum    0.112
#> female     0.000
#> black      0.000
#> exper     -0.096

```

6 Weighting

6.1 IPW by hand

```
e <- gss$ps
D <- gss$D
Y <- gss$Y

## Horvitz-Thompson
ate_ht <- mean(D * Y / e) - mean((1 - D) * Y / (1 - e))

## Hajek / stabilized -- divide by the sum of the weights
w1 <- D / e
w0 <- (1 - D) / (1 - e)
ate_sipw <- sum(w1 * Y) / sum(w1) - sum(w0 * Y) / sum(w0)

c(horvitz_thompson = ate_ht, stabilized = ate_sipw)
#> horvitz_thompson      stabilized
#>           1.5371           0.6023
```

```
w <- ifelse(D == 1, 1 / e, 1 / (1 - e))
ess <- sum(w)^2 / sum(w^2)
c(max_weight = max(w), ess = ess, n = length(w), ess_share = ess / length(w))
#> max_weight      ess      n  ess_share
#>   105.4685  1241.7073 3509.0000    0.3539
```

The **effective sample size** is what your standard errors should reflect. If a handful of observations carry most of the weight, you have far less information than n suggests.

6.2 ATT and overlap weights

```
att_w <- ifelse(D == 1, 1, e / (1 - e))      # ATT weights
ato_w <- ifelse(D == 1, 1 - e, e)           # overlap weights

wmean_diff <- function(wt) {
  sum(wt * Y * D) / sum(wt * D) - sum(wt * Y * (1 - D)) / sum(wt * (1 - D))
}

c(ATE_stabilized = ate_sipw,
  ATT = wmean_diff(att_w),
  ATO_overlap = wmean_diff(ato_w))
#> ATE_stabilized      ATT      ATO_overlap
#>           0.6023      0.6310      0.6371
```

Three different numbers, because they are three different **estimands**, not three attempts at one. The overlap-weighted estimate describes the subpopulation where comparison is most informative, and its weights are bounded by construction.

6.3 The same thing as a weighted regression

The coefficient on a dummy in weighted least squares is the difference of the weighted means, so `lm(Y ~ D, weights = w)` must reproduce each Hájek estimate exactly. It also hands us a standard error: a sandwich (HC1), which treats the weights as known. Nothing here accounts for the propensity score having been estimated; the bootstrap in section 7 does.

```
w_ate <- ifelse(D == 1, 1 / e, 1 / (1 - e))
## the argument is called wvec, not wt: gss has a survey-weight column named
## wt,
## and lm() would quietly pick up that column instead of our vector
wls <- function(wvec) {
  f <- lm(Y ~ D, data = gss, weights = wvec)
  c(WLS = unname(coef(f)["D"]), se_HC1 = unname(sqrt(vcovHC(f, type =
    "HC1")["D", "D"])))
}
wls_tab <- rbind(ATE = wls(w_ate), ATT = wls(att_w), ATO = wls(ato_w))
cbind(by_hand = c(ate_sipw, wmean_diff(att_w), wmean_diff(ato_w)), wls_tab)
#>      by_hand      WLS se_HC1
#> ATE  0.6023 0.6023 0.04018
#> ATT  0.6310 0.6310 0.03636
#> ATO  0.6371 0.6371 0.03398
```

The `by_hand` and `WLS` columns agree to every printed digit, as they must.

6.4 Does weighting actually balance the covariates?

```
bal.tab(D ~ pareduc + wordsum + female + black + otherace + exper,
  data = gss, weights = ifelse(D == 1, 1 / e, 1 / (1 - e)),
  method = "weighting", un = TRUE, thresholds = c(m = 0.1))
#> Balance Measures
#>      Type Diff.Un Diff.Adj      M.Threshold
#> pareduc Contin.  0.736   -0.218 Not Balanced, >0.1
#> wordsum Contin.  0.793   -0.077  Balanced, <0.1
#> female  Binary  0.055   -0.027  Balanced, <0.1
#> black   Binary -0.053    0.026  Balanced, <0.1
#> otherace Binary -0.016    0.021  Balanced, <0.1
#> exper   Contin. -0.420    0.006  Balanced, <0.1
#>
#> Balance tally for mean differences
```

```

#>                                count
#> Balanced, <0.1                    5
#> Not Balanced, >0.1                1
#>
#> Variable with the greatest mean difference
#> Variable Diff.Adj                M.Threshold
#> pareduc    -0.218 Not Balanced, >0.1
#>
#> Effective sample sizes
#>                Control Treated
#> Unadjusted    2068  1441.
#> Adjusted      1623   410.4

```

7 G-computation and AIPW

```

## fit separate outcome models in each arm, then impute both potential outcomes
out_form <- Y ~ pareduc + wordsum + female + black + otherace + exper +
  I(exper^2)
m1 <- lm(out_form, data = gss[D == 1, ])
m0 <- lm(out_form, data = gss[D == 0, ])

mu1 <- predict(m1, newdata = gss)
mu0 <- predict(m0, newdata = gss)

ate_gcomp <- mean(mu1 - mu0)
ate_gcomp
#> [1] 0.6318

```

```

aipw_terms <- (mu1 - mu0) +
  D * (Y - mu1) / e -
  (1 - D) * (Y - mu0) / (1 - e)

ate_aipw <- mean(aipw_terms)

## the influence-function standard error comes free
se_aipw <- sd(aipw_terms) / sqrt(length(aipw_terms))

c(estimate = ate_aipw, se = se_aipw,
  lower = ate_aipw - 1.96 * se_aipw, upper = ate_aipw + 1.96 * se_aipw)
#> estimate      se      lower      upper

```



```
#> 0.60512 0.03981 0.52709 0.68315
```

Tip

The AIPW estimator is an average of n i.i.d. terms, so its standard error is just the standard deviation of those terms over $\sqrt{n}$. That is not a coincidence, `aipw_terms` is the *efficient influence function*, and this is the pattern that Week 13 generalizes.

7.1 Bootstrapping the whole procedure

Standard errors must account for the fact that the propensity score was *estimated*. The safest route is to bootstrap the entire pipeline.

```
aipw_fun <- function(data, idx) {  
  d <- data[idx, ]  
  psm <- glm(D ~ pareduc + wordsum + female + black + otherace +  
             exper + I(exper^2), data = d, family = binomial)  
  ee <- fitted(psm)  
  f1 <- lm(out_form, data = d[d$D == 1, ]); f0 <- lm(out_form, data = d[d$D ==  
0, ])  
  m1p <- predict(f1, newdata = d); m0p <- predict(f0, newdata = d)  
  mean((m1p - m0p) + d$D * (d$Y - m1p) / ee -  
       (1 - d$D) * (d$Y - m0p) / (1 - ee))  
}  
set.seed(723)  
bo <- boot(gss, aipw_fun, R = 500)  
c(estimate = bo$t0, boot_se = sd(bo$t), analytic_se = se_aipw)  
#>      estimate      boot_se analytic_se  
#>      0.60512      0.03579      0.03981
```

The two standard errors agree closely, which is reassuring: the influence-function formula is doing its job.

8 Sensitivity analysis

All of the above assumed we measured every confounder. We did not. So ask instead: **how strong would an unobserved confounder have to be?** Two answers, both computed from the long regression `reg` and both on the slides.

8.1 Oster's coefficient-stability bound

Oster (2019) starts from the old habit of comparing the coefficient *without* controls to the coefficient *with* them. A small move is reassuring only if the controls explained a good share of

the outcome; otherwise unobservables that explain more could still move the coefficient a long way. Her method judges the move relative to the gain in R^2 and extrapolates. The `robomit` package does the arithmetic (`install.packages("robomit")`).

```
library(robomit)
## two inputs. R2max: how much of Y all confounders together could explain;
Oster's rule
## of thumb is 1.3 x the R^2 of the regression with controls. delta: how
important the
## unobservables are relative to the controls (1 = equally important)
r2_long <- summary(reg)$r.squared
con <- "pareduc + wordsum + female + black + otherace + exper + I(exper^2)" ##
the controls

## beta*: the effect if the unobservables were as important as the controls
(delta = 1)
o_beta(y = "Y", x = "D", con = con, delta = 1, R2max = 1.3 * r2_long, type =
"lm", data = gss)
#> # A tibble: 10 x 2
#>   Name                                Value
#>   <chr>                                <dbl>
#> 1 beta*                                0.577
#> 2 (beta*-beta controlled)^2            0.00276
#> 3 Alternative Solution 1                7.02
#> 4 (beta[AS1]-beta controlled)^2        40.8
#> 5 Uncontrolled Coefficient             0.683
#> 6 Controlled Coefficient               0.629
#> 7 Uncontrolled R-square                0.130
#> 8 Controlled R-square                  0.243
#> 9 Max R-square                        0.316
#> 10 delta                               1

## delta*: how many times as important as the controls the unobservables would
have to be
## to drive the effect to zero
o_delta(y = "Y", x = "D", con = con, beta = 0, R2max = 1.3 * r2_long, type =
"lm", data = gss)
#> # A tibble: 7 x 2
#>   Name                                Value
#>   <chr>                                <dbl>
#> 1 delta*                               2.58
#> 2 Uncontrolled Coefficient             0.683
#> 3 Controlled Coefficient               0.629
```

```
#> 4 Uncontrolled R-square      0.130
#> 5 Controlled R-square       0.243
#> 6 Max R-square              0.316
#> 7 beta hat                  0
```

Read the first row of each table. Oster's standard: the finding is robust if the interval from the controlled coefficient to **beta*** excludes zero and **delta*** is above one. Here **beta*** is 0.58 and **delta*** is 2.6: unobservables would have to matter more than twice as much as everything we controlled for to erase the BA premium.

8.2 Partial R^2 and the robustness value

```
sens <- sensemakr(model = reg, treatment = "D",
                  benchmark_covariates = "wordsum",
                  kd = 1:3)

sens
#> Sensitivity Analysis to Unobserved Confounding
#>
#> Model Formula: Y ~ D + pareduc + wordsum + female + black + otherace + exper
+
#>      I(exper^2)
#>
#> Null hypothesis: q = 1 and reduce = TRUE
#>
#> Unadjusted Estimates of ' D ':
#>   Coef. estimate: 0.629
#>   Standard Error: 0.032
#>   t-value: 19.98
#>
#> Sensitivity Statistics:
#>   Partial R2 of treatment with outcome: 0.102
#>   Robustness Value, q = 1 : 0.285
#>   Robustness Value, q = 1 alpha = 0.05 : 0.262
#>
#> For more information, check summary.
```

```
plot(sens)
```

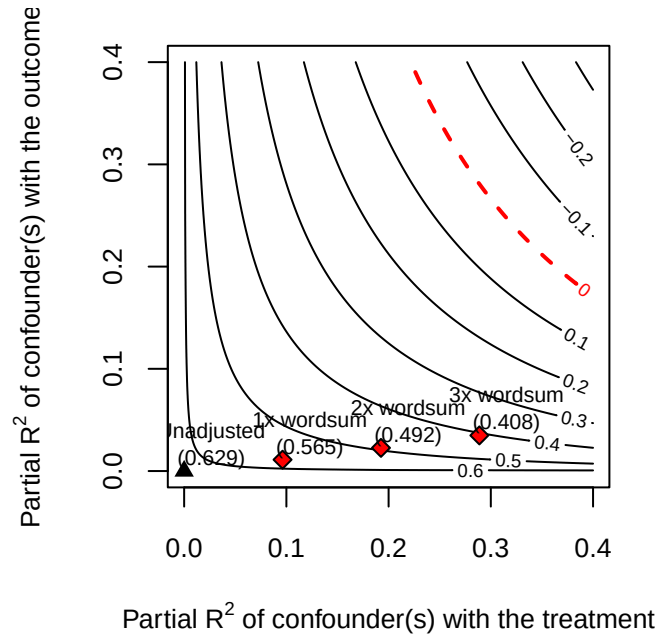


Figure 4: Contours of the adjusted estimate as a function of how much residual variation in treatment and outcome an unobserved confounder explains. Benchmarks show multiples of the vocabulary score.

Read the **robustness value**: it is the share of residual variation in both the treatment and the outcome that a confounder would need to explain to reduce the estimate to zero. Compare it to the benchmark: if a confounder three times as strong as measured verbal ability would be needed, the finding is reasonably robust; if half of one would do it, it is not.

9 Exercises

1. **Estimand discipline.** Redo the analysis targeting the **ATT** throughout (matching, ATT weights, and an ATT version of AIPW). Do the answers move? Explain why the ATE and ATT differ here in terms of who is on the margin of completing college.
2. **Trimming.** Restrict to $0.1 < \hat{e} < 0.9$ and recompute IPW and AIPW. How much does the estimate move, and how many observations did you drop? Write one sentence stating what population your new estimate describes.
3. **Deliberate misspecification, on simulated data.** Double robustness cannot be “confirmed” against observational data, because the causal truth is unknown. Simulate a world with a known ATE, then fit (a) a correct outcome model with a badly wrong propensity model and (b) the reverse. Confirm that AIPW stays close to the truth in both cases while IPW and g-computation each fail in one. This is double robustness in action.
4. **A bad control.** Add **prestige** (occupational prestige) to the covariate set. It is plausibly a *consequence* of holding a degree. Report how the estimate changes and explain, using the Week 5 vocabulary, why the new number answers a different question.

5. **Balance versus fit.** Compute the AUC of the propensity model. Then add interactions until balance improves. Does the AUC go up or down? Discuss what this means for the “propensity score paradox” from lecture.
6. **Effective sample size.** Plot the IPW weights against $\hat{e}$. Identify the ten most heavily weighted observations. Refit without them, how much does the estimate move relative to its standard error?

10 Session information

```
## R version and the packages this lab loaded
c(R = R.version.string, sapply(sessionInfo()$otherPkgs, \(p)
as.character(p$Version)))
#>               R               robomit
#> "R version 4.5.1 (2025-06-13)"      "1.0.7"
#>           Matching              MASS
#> "4.10-15"              "7.3-65"
#>           boot              lmtest
#> "1.3-32"              "0.9-40"
#>           zoo              sandwich
#> "1.8-14"              "3.1-1"
#>           sensemakr              cobalt
#> "0.1.6"              "5.0.0"
#>           MatchIt              ggplot2
#> "4.8.1"              "4.0.0"
#>           dplyr
#> "1.1.4"
```